

ESP32 4 — HTTP POST and a local Python server

Tutorial Group: ESP32

Goal

In this tutorial, you will make the ESP32 communicate with a computer over Wi-Fi using HTTP. The ESP32 will act as the client and your computer will act as the server. The ESP32 will send a simple HTTP POST containing “Hello World,” and the computer will receive and display it.

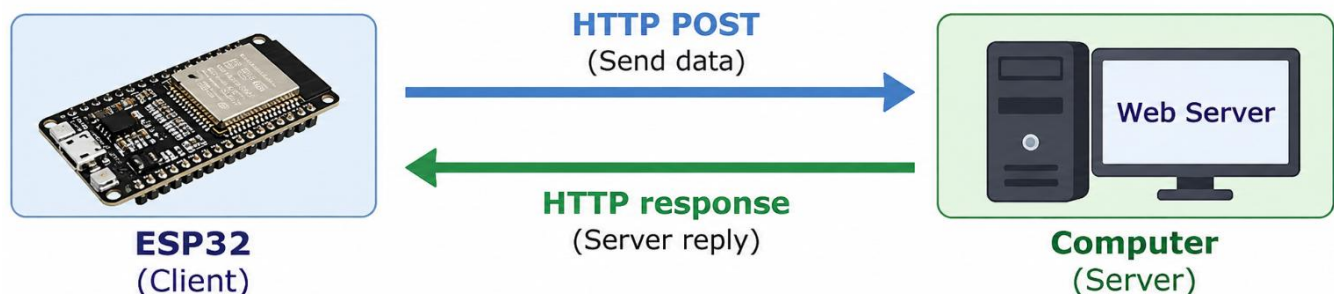
This tutorial introduces client/server communication, HTTP POST, running a simple Python web server, finding the computer's IP address, and sending information from an ESP32 to a computer.

What You Need

- ESP32-WROOM-32
- USB cable
- Computer with the Arduino IDE and Python 3
- Wi-Fi network

Part 1 — Client and Server

The ESP32 is the client because it initiates communication. The computer is the server because it waits for requests and responds.



Part 2 — Install Python

If Python 3 is not already installed, install a current Python 3 release. On Windows, select “Add Python to PATH” during installation.

Verify the installation from a command prompt:

```
python --version
```

If that command does not work, try:

```
py --version
```

Part 3 — Create the Python Server

Create a file named `esp32_server.py` and enter:

```
from http.server import BaseHTTPRequestHandler, HTTPServer

class ESP32Handler(BaseHTTPRequestHandler):
```

```

def do_POST(self):
    length = int(self.headers.get('Content-Length', 0))
    data = self.rfile.read(length)

    print()
    print("POST received:")
    print(data.decode("utf-8"))

    self.send_response(200)
    self.end_headers()
    self.wfile.write(b"Received")

server = HTTPServer(("0.0.0.0", 5000), ESP32Handler)

print("ESP32 server running")
print("Listening on port 5000")
print("Press Ctrl+C to stop")

server.serve_forever()

```

The 0.0.0.0 address tells the server to listen on the computer's available network interfaces rather than only one specific IP address.

This uses only Python's standard library; no additional Python packages are required.

Part 4 — Start the Server

Open a command prompt in the directory containing `esp32_server.py` and run:

```
python esp32_server.py
```

Leave this window running. Your computer is now acting as the server.

Part 5 — Find the Computer's IP Address

On Windows, run:

```
ipconfig
```

Find the IPv4 address for the Wi-Fi adapter. For example, it might be 192.168.1.50. Your address will probably be different.

Network troubleshooting:

- The ESP32-WROOM-32 and the computer must be able to reach each other over the network
- If the ESP32 cannot connect to the Python server, check that both devices are on the same Wi-Fi network and that the computer's firewall allows incoming TCP connections to port 5000
- Some managed or public Wi-Fi networks prevent wireless devices from communicating directly with each other. If both devices are connected to the same network but the ESP32 cannot reach the computer, the network may have client isolation enabled.

Part 6 — Create the ESP32 Program

```
#include <WiFi.h>
```

```

#include <HTTPClient.h>

const char* ssid = "YourNetworkName";
const char* password = "YourPassword";

const char* serverURL = "http://192.168.1.50:5000/";

void setup()
{
    Serial.begin(115200);

    WiFi.begin(ssid, password);

    while (WiFi.status() != WL_CONNECTED)
    {
        Serial.print(".");
        delay(500);
    }

    Serial.println(" Wi-Fi connected");
    Serial.println(WiFi.localIP());

    HTTPClient http;
    http.begin(serverURL);

    int responseCode = http.POST("Hello World");

    Serial.print("HTTP Response: ");
    Serial.println(responseCode);

    http.end();
}

void loop()
{
    // All of the work in this tutorial occurs once in setup().
}

```

Replace the network information and server IP address with your own values.

Part 7 — Test the Communication

Upload the program and open the Serial Monitor at 115200 baud. The Python server should display:

```

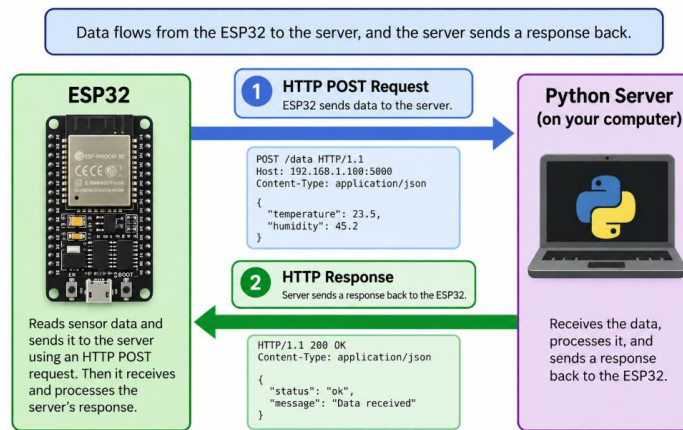
POST received:
Hello World

```

The ESP32 should also display an HTTP response code. A response of 200 indicates that the server successfully processed the request.

Part 8 — What Happened?

The ESP32 initiated the transaction, but the server also sent a response. The communication therefore occurs in both directions even though the ESP32 started it:



What You Should Understand

- The difference between an HTTP client and server.
- How an ESP32 can send information to a computer using HTTP POST.
- How to start and use a simple Python server.
- How to find the computer's IP address on the local network.
- How to recognize a successful HTTP response.

Before Moving On

You should be able to answer **yes** to all of the following:

- ☐ I can install and verify Python 3.
- ☐ I can start the supplied Python server.
- ☐ I can find my computer's IP address.
- ☐ I can configure the ESP32 to contact my computer.
- ☐ I can send an HTTP POST.
- ☐ I can identify a successful HTTP response.
- ☐ I understand the client/server relationship.

The Python server created in this tutorial provides a simple local HTTP POST endpoint for receiving ESP32 data.